EECS 489 - WN 25
Discussion 3

Assignment 1 Recap, DNS, select() Demo

Logistics

Assignment 2:

- Assignment 2 released by this Friday, due Friday before Spring Break (Feb 28).
- Autograder will be released by end of next week.

Office Hours:

- Staff add office hours around project deadlines! Please add the course Google calendar (linked here, also on the syllabus).

Discussions:

- One GSI will teach all three discussions in a given week – all three will be identical.
- Friday 12:30 discussions are recorded.

About me

- I’m Aditya (singhvi@umich.edu)
- SUGS CSE Student
- I may have taught you before! IA for EECS 376 (FA22-WN23) and EECS 477 (FA23-WN24)
- Second semester teaching EECS 489
- From Bay Area, CA and Pune, India
- Office Hours: Tuesday, 10am-12pm in BBB LC
- Talk to me about:
  - Football/Basketball/Cricket
  - International student things
  - Jobs/IAing/etc.

Assignment 1 Recap

Assignment 1: Common Mistakes

Confusion between **bytes** and **bits**

- Conventionally, rates are given in ___**bits** per second, while actual units of data are most common as ___**bytes**.
- Historical Reasoning: The network doesn't care how bits are grouped together, but you need an entire byte to make sense of the data.

Assignment 1: Common Mistakes

Not calling send() in a loop!

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

Sends the message described by **buf** and **len** over the connection described by sockfd.

- **Server:** `sockfd` should be the `connfd` (from accept) used to communicate with the client.
- **Client:** `sockfd` can just be the actual `sockfd`.

Returns number of bytes actually sent; this may not be all the bytes in the message. -1 on error.

```
inline int send_data(int connectionfd, const char *message,
                     size_t message_len) {
    size_t sent = 0;
    do {
        const ssize_t n =
            send(connectionfd, message + sent, message_len - sent, 0);
        sent += n;
    } while (sent < message_len);
    return 0;
}
```

Assignment 1: Common Mistakes

Not using the MSG_WAIT_ALL flag for recv!

```
ssize_t recv(int sockfd, const void * buf, size_t len, int flags);

// Example
ssize_t bytes_recvd = recv(sockfd, buffer, MSG_SIZE, 0);
// here bytes_recvd is not always == to MSG_SIZE

// But we also can do
ssize_t bytes_recvd = recv(sockfd, buffer, MSG_SIZE,
MSG_WAITALL);
// here bytes_recvd is == MSG_SIZE, but we block until then
```

Case Study: Network Buffer Backlog

- Many students had errors like this one:

```
2025-01-28 04:43:16.232 | ERROR    | working_dir.test_all:assert_with_logging:21 - Assertion failed: Client did not run for expected amount of time
2025-01-28 04:43:16.290 | INFO     | working_dir.test_all:create_server:180 -
---- Server Output ----
 [2025-01-28 04:43:00.055] [info] iPerfer server started
[2025-01-28 04:43:00.159] [info] Client connected
[2025-01-28 04:43:16.231] [info] Received=2000 KB, Rate=0.996 Mbps, RTT=0 ms
2025-01-28 04:43:16.291 | INFO     | working_dir.test_all:create_client:148 -
---- Client Output ----
 [2025-01-28 04:43:02.195] [info] Sent=2000 KB, Rate=8.000 Mbps, RTT=0 ms

```

- Observe:
  - RTT is calculated correctly!
  - Server is running for 16 seconds (should be 2) but calculating rate correctly
  - Client is running for 2 seconds (correct) but calculating rate as 8x the expected

Case Study: Network Buffer Backlog

```
2025-01-28 04:43:16.232 | ERROR    | working_dir.test_all:assert_with_logging:21 - Assertion failed: Client did not run for expected amount of time
2025-01-28 04:43:16.290 | INFO     | working_dir.test_all:create_server:180 -
---- Server Output ----
 [2025-01-28 04:43:00.055] [info] iPerfer server started
[2025-01-28 04:43:00.159] [info] Client connected
[2025-01-28 04:43:16.231] [info] Received=2000 KB, Rate=0.996 Mbps, RTT=0 ms
2025-01-28 04:43:16.291 | INFO     | working_dir.test_all:create_client:148 -
---- Client Output ----
 [2025-01-28 04:43:02.195] [info] Sent=2000 KB, Rate=8.000 Mbps, RTT=0 ms

```

Typically caused by one of the following:

- Not waiting for server ACKs before sending more data
- Not sending in a loop (aka not sending all the data)
- Not receiving with the MSG_WAITALL flag

What do these have in common?

Case Study: Network Buffer Backlog

```
2025-01-28 04:43:16.232 | ERROR    | working_dir.test_all:assert_with_logging:21 - Assertion failed: Client did not run for expected amount of time
2025-01-28 04:43:16.290 | INFO     | working_dir.test_all:create_server:180 -
---- Server Output ----
 [2025-01-28 04:43:00.055] [info] iPerfer server started
[2025-01-28 04:43:00.159] [info] Client connected
[2025-01-28 04:43:16.231] [info] Received=2000 KB, Rate=0.996 Mbps, RTT=0 ms
2025-01-28 04:43:16.291 | INFO     | working_dir.test_all:create_client:148 -
---- Client Output ----
 [2025-01-28 04:43:02.195] [info] Sent=2000 KB, Rate=8.000 Mbps, RTT=0 ms

```

When you send more data than your system can handle, or don't wait for your system to actually send all the data, you create **backlogs in your network buffer.**

Data sending continues even after the client process dies (e.g. for 14 extra seconds).

Case Study: iPerf and iPerfer Differences

- Several students noticed differences between **iPerf** and **iPerfer** when running measurements for Part 4.
- Suppose **h1** and **h2** are communicating while **h3** and **h4** are communicating, all **sharing a single link of 20 Mbps.**
- How much bandwidth should each of them get?

Case Study: iPerf and iPerfer Differences

- Suppose **h1** and **h2** are communicating while **h3** and **h4** are communicating, all **sharing a single link of 20 Mbps.**
- **iPerf** might measure (correctly)
  - **h1-h2:** 13 Mbps
  - **h3-h4:** 7 Mbps
- **iPerfer** might measure:
  - **h1-h2:** 20 Mbps
  - **h3-h4:** 20 Mbps
- **Why?**

Case Study: iPerf and iPerfer Differences

- Suppose **h1** and **h2** are communicating while **h3** and **h4** are communicating, all **sharing a single link of 20 Mbps.**
- **iPerf** might measure (correctly)
  - **h1-h2:** 13 Mbps
  - **h3-h4:** 7 Mbps
- **iPerfer** might measure:
  - **h1-h2:** 20 Mbps
  - **h3-h4:** 20 Mbps
- **iPerfer does not saturate the link; iPerf does.**

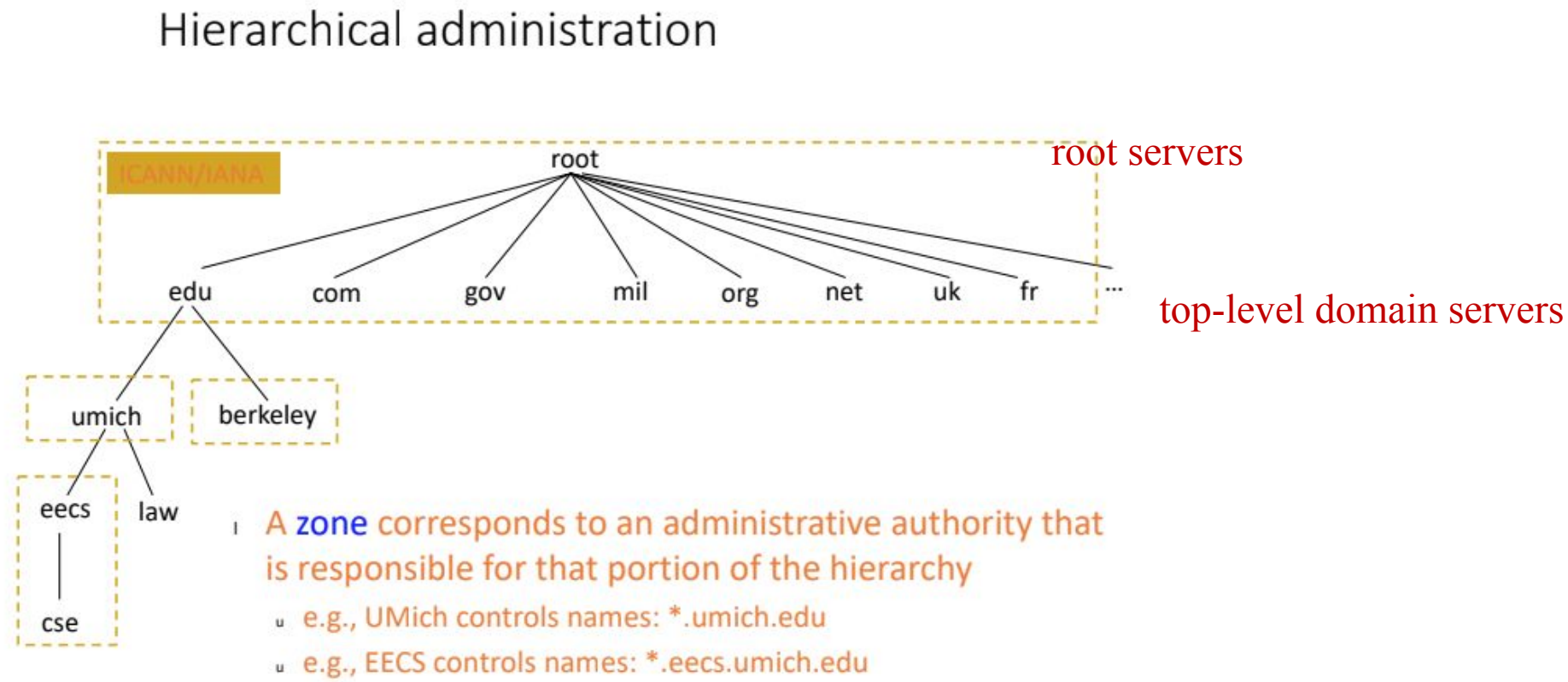
Assignment 1: Takeaways

- **Come to discussions.** Socket programming was discussed extensively in Weeks 1, 2, including the tips on sending in a loop and MSG_WAITALL.
- **Read the documentation.** Network functions have been around for a while, and have very clear man pages, as well as thousands of use cases online.
- **Write clean code.** Many bugs (e.g. due to unit conversion) were caused by messy, unreadable code. Abstract common behavior into helper functions, design in a modular way, and take advantage of modern C++ constructs.
- **Start early!**

DNS Review

DNS: Recap

Domain Name System to map URLs (like docs.google.com) to machine addresses (like 172.217.4.46).



root servers

top-level domain servers

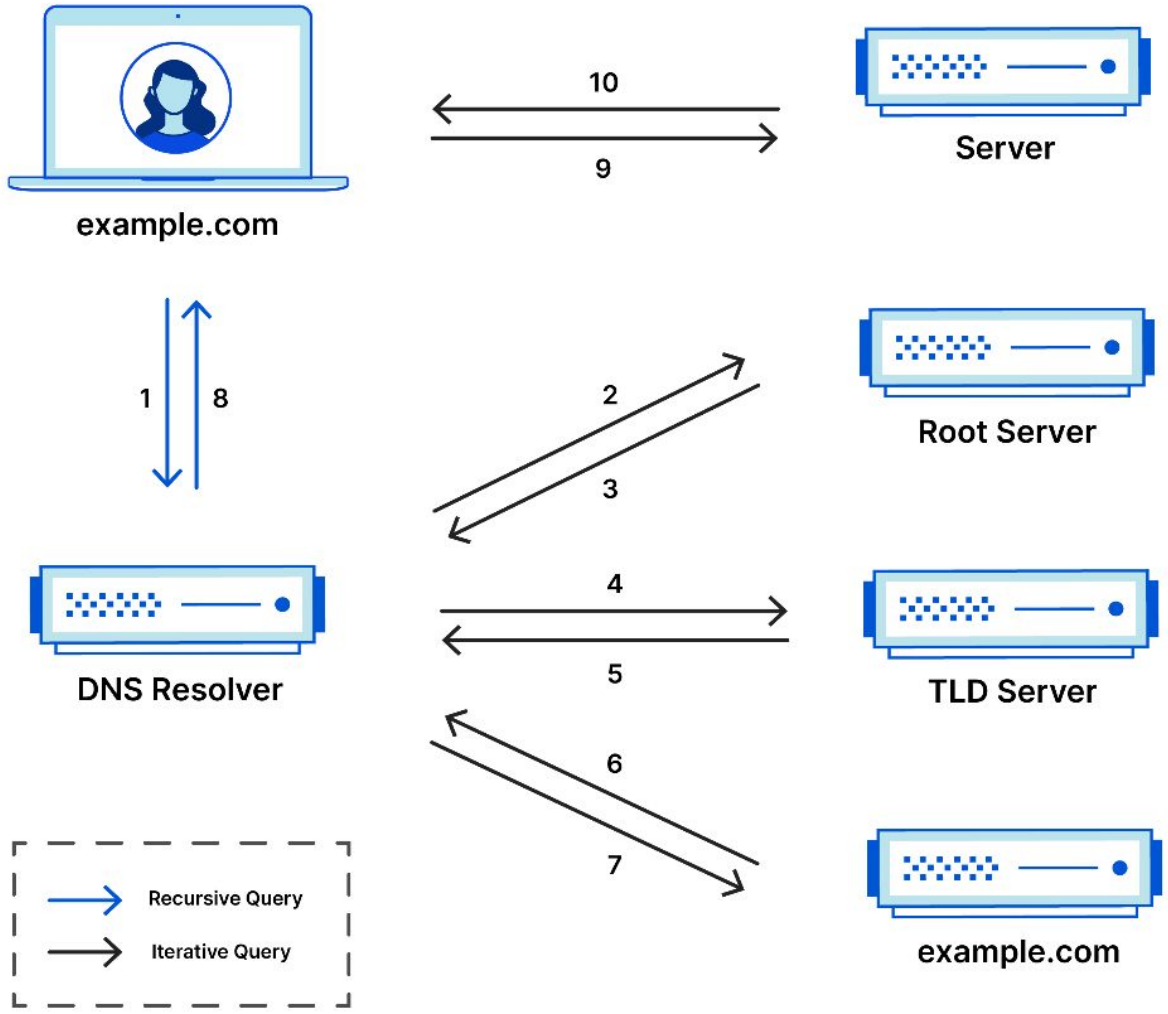
authoritative DNS servers

DNS: Protocol

1. Host sends a **query** to local DNS server
   a. *We learn later in the semester how a host knows where to find local DNS server*
   b. Asks a question like: **What is the IP address of docs.google.com?**
2. Local DNS Server either:
   a. Has the reply cached – someone else in the network looked this up recently, so it can just respond directly.
   b. Does an iterative/recursive name resolution
      i. Might skip some steps if it has other things cached!
3. Eventually, we get a **response** from an **authoritative DNS server**, which has an A-type record mapping the hostname to the IP address
4. This response propagates back to our host, who can then remember this IP address and send IP packets to the address!

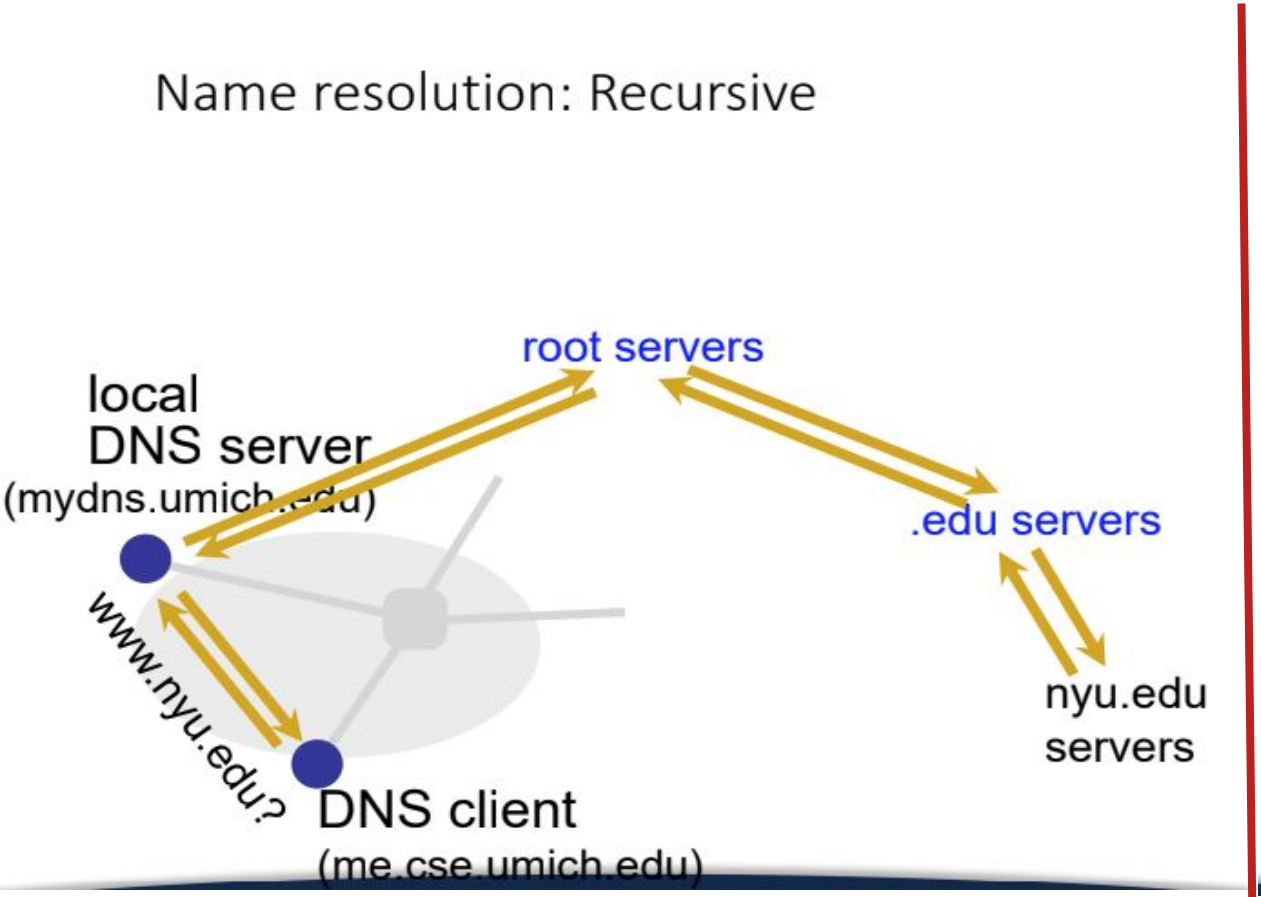
DNS: Protocol

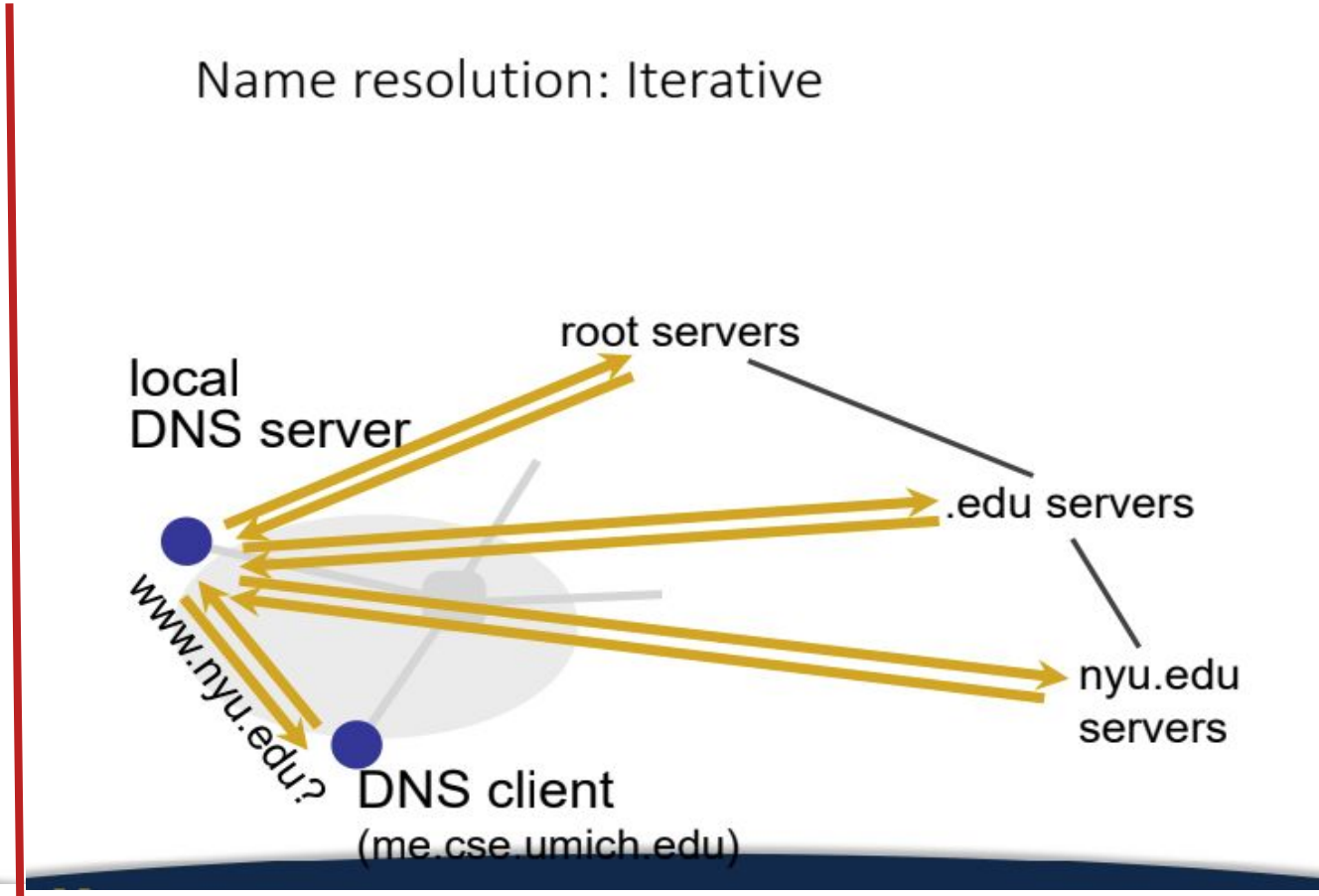
Reference: https://www.cloudflare.com/learning/dns/what-is-dns/



DNS: Seeing it yourself!

DNS: Recursive vs. Iterative





Lecture Based Questions - Q1

Suppose the UM EECS Department has a DNS server for all computers in the department.

How could you determine if an external web site was likely to be accessed from another computer in EECS a couple of seconds ago?

Lecture Based Questions - Q1

Suppose EECS has a DNS server for all computers in the department.

How could you determine if an external web site was likely to be accessed from another computer in EECS a couple of seconds ago?

Perform consecutive dig queries and compare the query time.

Lecture Based Questions - Q1

Suppose EECS has a DNS server for all computers in the department.

How could you determine if an external web site was likely to be accessed from another computer in EECS a couple of seconds ago?

Perform consecutive dig queries and compare the query time.

If the website was recently accessed, the first dig query would be very fast, as the address would be cached. On the other hand, subsequent queries (after a while of nobody accessing the website) might be much slower.

Lecture Based Questions - Q2

Suppose you are trying to access the page web.eecs.umich.edu/course/eecs489. You are connected to your home WiFi with its own local DNS (from your Internet Service Provider), and are not connected to the UM network. What is the order of DNS servers queried over time?

Assumptions:

- No prior caching, **iterative** name resolution
- umich.edu and eecs.umich.edu are in separate zones
- eecs.umich.edu is authoritative for all hostnames ending in .eecs.umich.edu

Lecture Based Questions - Q2

Suppose you are trying to access the page web.eecs.umich.edu/course/eecs489. You are connected to your home WiFi with its own local DNS (from your Internet Service Provider), and are not connected to the UM network. What is the order of DNS servers queried over time?

Assumptions:

- No prior caching, **iterative** name resolution
- umich.edu and eecs.umich.edu are in separate zones
- eecs.umich.edu is authoritative for all hostnames ending in .eecs.umich.edu

1. Your Device → DNS Resolver
2. DNS Resolver → root nameserver → DNS Resolver
3. DNS Resolver → .edu nameserver → DNS Resolver
4. DNS Resolver → umich.edu nameserver → DNS Resolver
5. DNS Resolver → eecs.umich.edu nameserver → DNS Resolver
6. DNS Resolver → Your Device

Lecture Based Questions - Q3

Suppose you are trying to access the page web.eecs.umich.edu/course/eecs489. You are connected to your home WiFi with its own local DNS (from your Internet Service Provider), and are not connected to the UM network. What is the order of DNS servers queried over time?

Assumptions:

- No prior caching, **recursive** name resolution
- umich.edu and eecs.umich.edu are in separate zones
- eecs.umich.edu is authoritative for all hostnames ending in .eecs.umich.edu

Lecture Based Questions - Q3

Suppose you are trying to access the page web.eecs.umich.edu/course/eecs489. You are connected to your home WiFi with its own local DNS (from your Internet Service Provider), and are not connected to the UM network. What is the order of DNS servers queried over time?

Assumptions:

- No prior caching, **recursive** name resolution
- umich.edu and eecs.umich.edu are in separate zones
- eecs.umich.edu is authoritative for all hostnames ending in .eecs.umich.edu

1. DNS Resolver → root nameserver
2. root nameserver → .edu nameserver
3. .edu nameserver → umich.edu nameserver
4. umich.edu nameserver → eecs.umich.edu nameserver
5. eecs.umich.edu nameserver → umich.edu nameserver
6. umich.edu nameserver → .edu nameserver
7. .edu nameserver → root nameserver
8. root nameserver → DNS Resolver

Lecture Based Questions - Q4

What is a CNAME record, and why is it useful?

Lecture Based Questions - Q4

What is a CNAME record, and why is it useful?

A CNAME (Canonical Name) record aliases one domain name to another. This allows us to point subdomains to their actual domain.

```
16:32:34 adityasinghvi ~ $ dig cse.umich.edu

; <<>> DiG 9.10.6 <<>> cse.umich.edu
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 46881
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;cse.umich.edu.         IN  A

;; ANSWER SECTION:
cse.umich.edu.      1800    IN  CNAME   cse.eecs.umich.edu.
cse.eecs.umich.edu. 900 IN  A   141.212.113.143

;; Query time: 334 msec
;; SERVER: 1.1.1.1#53(1.1.1.1)
;; WHEN: Wed Sep 18 16:32:38 EDT 2024
;; MSG SIZE  rcvd: 81
```

Lecture Based Questions - Q4

What is a CNAME record, and why is it useful?

A CNAME (Canonical Name) record aliases one domain name to another. This allows us to point subdomains to their actual domain.

A common use case is with CDNs:

- **Example:** Clicking on an Image URL on the NY Times website gets you something like https://static01.nyt.com/images/2024/09/18/…
- This eventually points to a fastly.net, which is a CDN! So, all requests to get images for NYT articles are actually redirected to the CDN, which can load-balance effectively.

```
🔥 16:36:06 adityasinghvi ~ $ dig static01.nyt.com

; <<>> DiG 9.10.6 <<>> static01.nyt.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 31638
;; flags: qr rd ra; QUERY: 1, ANSWER: 7, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;static01.nyt.com.      IN  A

;; ANSWER SECTION:
static01.nyt.com.   479 IN  CNAME   static.prd.map.nytimes.com.
static.prd.map.nytimes.com. 479 IN  CNAME   static.prd.map.nytimes.xovr.nyt.net.
static.prd.map.nytimes.xovr.nyt.net. 279 IN CNAME nytimes.map.fastly.net.
nytimes.map.fastly.net. 39  IN  A   151.101.1.164
nytimes.map.fastly.net. 39  IN  A   151.101.129.164
nytimes.map.fastly.net. 39  IN  A   151.101.65.164
nytimes.map.fastly.net. 39  IN  A   151.101.193.164

;; Query time: 23 msec
;; SERVER: 1.1.1.1#53(1.1.1.1)
;; WHEN: Wed Sep 18 16:36:41 EDT 2024
;; MSG SIZE  rcvd: 228
```

Lecture Based Questions - Q5

What are the consequences of a malicious entity gaining control over your local DNS server?

Lecture Based Questions - Q5

What are the consequences of a malicious entity gaining control over your local DNS server?

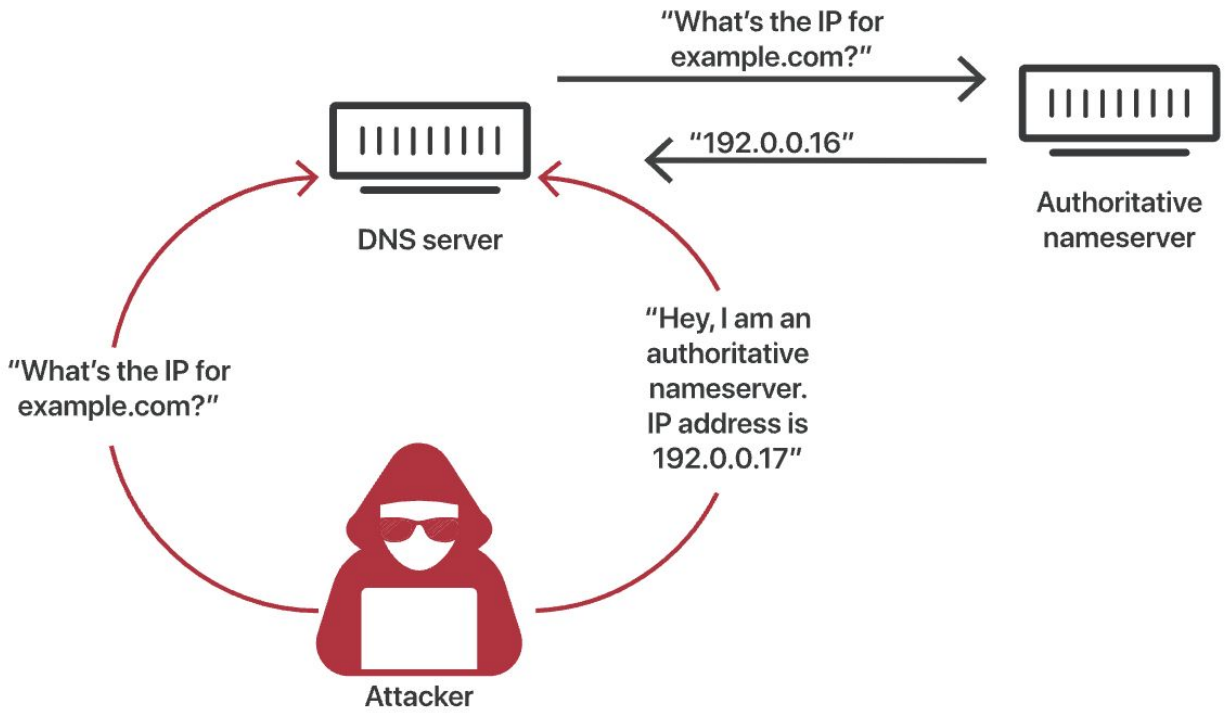
- Can direct you to malicious servers that resemble real servers
  - E.g. instead of accessing bankofamerica.com at its real IP address, your local DNS server tells you that it is actually a server controlled by the attacker that resembles the bank website
  - **DNS Cache Poisoning:** Attackers can make a DNS request to their local DNS server, and then spoof the response themselves before a real nameserver has a chance to respond. **This poisons the DNS cache for others using the same nameserver!**
- Can monitor your browsing (i.e. what hosts are you visiting) and block websites.
  - A lot of firewalls are implemented (partially) at the DNS level

Classic DNS Cache Poisoning Attack

- **Objective:** Poison cache of a recursive nameserver to map **goodsite.com** to attacker's IP
- **Challenge:** DNS only accepts responses to pending queries:
  - **Query ID** must match
  - **UDP port** must match
  - **Question field** (duplicated in response) must match
  - Reply must have **names within the same domain** as the question
- **Attacker Advantages**
  - Only need to get these fields right once (malformed packets are dropped)
  - First correct response wins

Classic DNS Cache Poisoning Attack

DNS Cache Poisoning Process:



Classic DNS Cache Poisoning Attack

1. Discovering the **source port** and **query ID**:
    a. Attacker sends a query to the victim recursive nameserver for an attacker-controlled domain name
    b. Learns UDP port used by server (typically does not change) and query ID (often was assigned sequentially) when query arrives at attacker's authoritative nameserver
2. Attacker sends a legitimate request for *goodsite.com* to the victim nameserver
3. Attacker floods the recursive nameserver with forged DNS replies
    a. Attacker knows the nameserver just sent out a query for *goodsite.com*
    b. Just needs correctly-forged reply to arrive faster than legitimate reply

DNS Cache Poisoning Defenses

- **Source Port Randomization:** Increases randomness from 16 bits → 32 bits
- **0x20 Encoding:** Randomize capitalization in domain name in the query. This has potential to increase randomness, but causes compatibility issues and isn't used often in practice.
- **Randomize Nameserver:** Typically adds around 2-3 bits of randomness in practice
- **DNSSEC:** Adds signatures to DNS records; strong but lacks wide adoption.

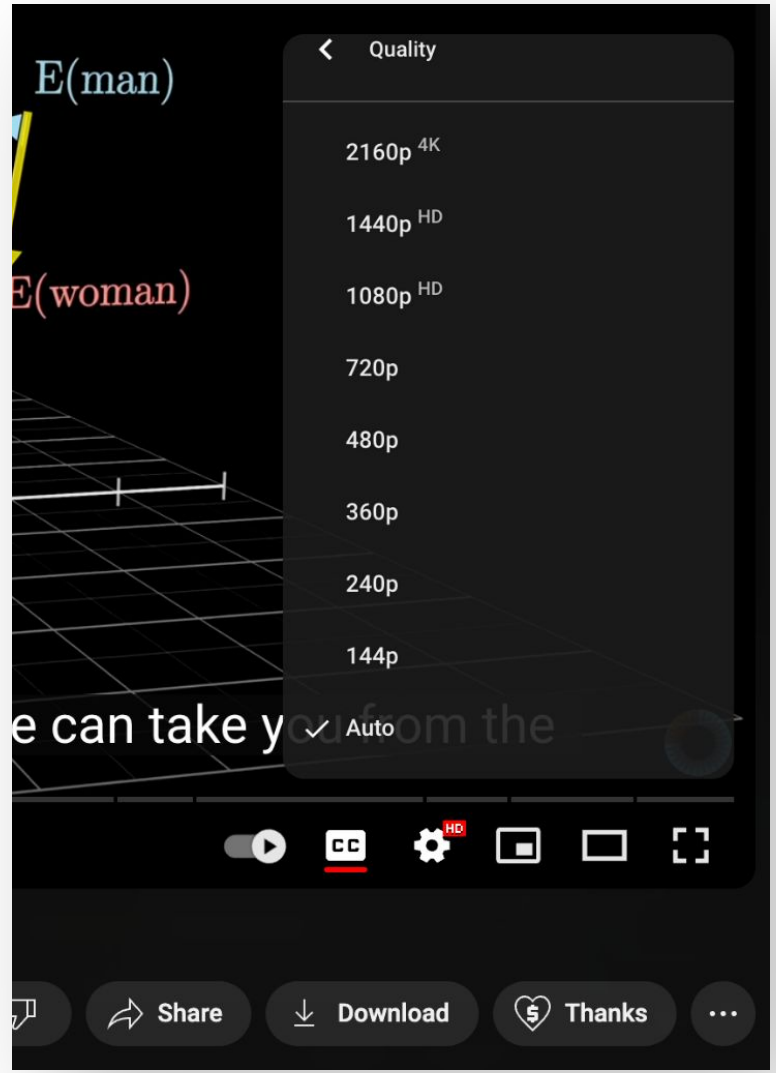
Want to learn more? Take **EECS 588** (and also EECS 388).

Assignment 2 Prep

Assignment 2

Part 1: HTTP Proxy for video streaming

- Sits between a client (web browser) and a standard video server.
- Modifies client requests to the video server to ask for the correct bitrate, based on constant measurement; echoes replies from the video server back to the client.
- Should be able to deal with multiple concurrent client connections!



Assignment 2

Part 2: Load Balancer

- Load-balances to different videoservers using one of two strategies:
  - **Strategy 1:** Round robin
  - **Strategy 2:** Geographical proximity
- Proxy calls the Load Balancer whenever a new client connects to figure out which videoserver to connect to!

Assignment 2

Integrating Part 1 and Part 2

- Have the HTTP proxy call the DNS server when a new client connects to figure out which server to route that client's traffic to.
- You can work on Part 1, Part 2 independently and then integrate at the end – both are testable on their own.

Assignment 2: Tips

- Pay attention to network/host byte order
- Ensure that the bugs from P1 don’t occur again by writing and using helper functions
- Understand select() code through Discussion examples
- Test as you build! You can first test a pass-through proxy before moving onto rate-adaptation.

A2 Prep

How would you modify the P1 server to handle multiple clients simultaneously?

A2 Prep

How would you modify the P1 server to handle multiple clients simultaneously?

- **Multithreading:** Create a separate process for each client and let the OS figure out how to run the threads to ensure fairness, etc.
  - Often simpler to code because you can use OS primitives for resource management
- **Polling with `select()`:** Have multiple file descriptors and continually "poll" them to see which ones need attention.
  - This can be much faster than multithreading if you have a lot of clients, as threads have a bunch of overhead
  - Can make programming more complex, and the parallelism has to be tightly integrated into the code.

A2 Prep: select()

```
#include <sys/select.h>
int select(int nfds,                    // highest # file descriptor + 1
           fd_set *readfds,             // fds to be checked for reading
           fd_set *writefds,            // fds to be checked for writing
           fd_set *exceptfds,           // fds to be checked for errors
           struct timeval *timeout);    // max wait time for anything to be ready

// For A2, we only care about readfds and have no timeout, so
// the invocation typically looks like this:
select(FD_SETSIZE, &readfds, NULL, NULL, NULL);
```

A2 Prep: select()

Some useful macros:

```
// Add fd to the set
void FD_SET(int fd, fd_set *set);
 // Remove fd from the set
void FD_CLR(int fd, fd_set *set);
// Return true if fd is in set, might not be
after select()
int FD_ISSET(int fd, fd_set *set);
// Clear all entries from set
void FD_ZERO(fd_set *set);
```

A2 Prep: select()

WARNING: select() can monitor only file descriptors numbers that are less than **FD_SETSIZE** (1024)—an unreasonably low limit for many modern applications—and this limitation will not change. All modern applications should instead use poll(2) or epoll(7), which do not suffer this limitation.

Note well: Upon return, each of the file descriptor sets is modified in place to indicate which file descriptors are currently "ready". Thus, if using **select**() within a loop, the sets *must be reinitialized* before each call.

A2 Prep: select() Demo

https://github.com/mosharaf/eecs489/tree/w25/Discussion/ds3-echoserver

A server that allows multiple clients to connect, and echoes client messages back.

Exercise: Implementing a heartbeat server

Note: Feel free to copy any demo code from Discussions for your projects!